Complexity_Analysis

Comprehensive Time & Space Complexity Analysis

1. Linear Data Structures

Arrays vs. Linked Lists

Data Structure	Operation	Time Complexity	Space Complexity	Notes
Array	Access (Index)	$\Theta(1)$	$O(N)$	Direct memory addressing.
	Search (Unsorted)	$\Theta(N)$	-	Linear scan.
	Search (Sorted)	$\Theta(\log N)$	-	Binary Search.
	Insert (Front/Middle)	$\Theta(N)$	-	Requires shifting elements.
	Insert (End)	$\Theta(1)$	-	Unless resize needed.
	Delete	$\Theta(N)$	-	Requires shifting elements.
Singly Linked List	Access (Index)	$\Theta(N)$	$O(N)$	Must traverse from Head.
	Search	$\Theta(N)$	-	Linear traversal.
	Insert (Front)	$\Theta(1)$	-	Update Head pointer.
	Insert (After Node)	$\Theta(1)$	-	*If pointer to node is known.*
	Insert (End)	$\Theta(N)$	-	$\Theta(1)$ if Tail pointer is maintained.
	Delete (Front)	$\Theta(1)$	-	Update Head pointer.
	Delete (End)	$\Theta(N)$	-	Must traverse to find 2nd to last node.
Doubly Linked List	Delete (Given Node)	$\Theta(1)$	$O(N)$	No traversal needed (has `Prev`).

Stacks & Queues

Algorithm	Operation	Time Complexity	Space Complexity	Notes
Stack	Push / Pop	$\Theta(1)$	$O(N)$	Top pointer movement.
	Peek	$\Theta(1)$	-	
Queue	Enqueue / Dequeue	$\Theta(1)$	$O(N)$	Front/Rear pointer movement.
Circular Queue	Enqueue / Dequeue	$\Theta(1)$	$O(N)$	Modulo arithmetic handles wrapping.

---

2. Tree Data Structures

Binary Search Tree (BST)

Operation	Average Case	Worst Case	Notes
Search	$O(\log N)$	$O(N)$	Worst case occurs in **Skewed Trees**.
Insert	$O(\log N)$	$O(N)$	Dependent on tree height (H).
Delete	$O(\log N)$	$O(N)$	Dependent on tree height (H).
Traversal	$O(N)$	$O(N)$	Must visit every node.
Space	$O(N)$	$O(N)$	Storing N nodes.

Huffman Coding (Compression)

- **Time Complexity:** $O(N \log N)$
  - N insertions into Priority Queue. Each insertion takes $\log N$.
- **Space Complexity:** $O(N)$ to store the tree.

---

3. Graph Algorithms

Representations

Representation	Space Complexity	Edge Check (u, v)	Iterating Neighbors	Best For
Adjacency Matrix	$O(V^2)$	$O(1)$	$O(V)$	Dense Graphs

Representation	Space Complexity	Edge Check (u, v)	Iterating Neighbors	Best For
Adjacency List	$O(V + E)$	$O(\text{degree of } u)$	$O(\text{degree of } u)$	Sparse Graphs

Traversals & Pathfinding

Algorithm	Time Complexity	Space Complexity	Logic / Notes
BFS	$O(V + E)$	$O(V)$	Uses **Queue**. Visits every node/edge once.
DFS	$O(V + E)$	$O(V)$	Uses **Stack/Recursion**.
Topological Sort	$O(V + E)$	$O(V)$	Kahn's Algorithm (In-Degree).
Warshall's Algo	$O(V^3)$	$O(V^2)$	Transitive Closure (All-pairs reachability).

Structural Analysis (DFS Extensions)

Algorithm	Time Complexity	Space Complexity	Notes
Articulation Points	$O(V + E)$	$O(V)$	Single DFS pass computing `d[]` and `low[]`.
Bridges	$O(V + E)$	$O(V)$	Similar logic to APs but strict inequality check.

---

4. General Complexity Concepts

Asymptotic Notations

- $O(n)$ **(Big O):** Upper Bound. Worst-case scenario.
- $\Omega(n)$ **(Big Omega):** Lower Bound. Best-case scenario.
- $\Theta(n)$ **(Big Theta):** Tight Bound. Exact growth rate.

Growth Rates (Fastest to Slowest)

1. $O(1)$ - Constant
2. $O(\log n)$ - Logarithmic (Binary Search, Balanced Trees)
3. $O(n)$ - Linear (Loops, Traversals)

4. $O(n \log n)$ - Linearithmic (Merge Sort, Heap Sort)
5. $O(n^2)$ - Quadratic (Nested Loops, Bubble Sort)
6. $O(2^n)$ - Exponential (Recursive Fibonacci)

---

5. Practice Problems

Set A: Calculate Time Complexity ($T(n)$)

Problem 1: Simple Loop

```
int count = 0;
for (int i = 0; i < n; i += 2) {
    count++;
}
```

Problem 2: Dependent Nested Loops

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        cout << i << j;
    }
}
```

Problem 3: Logarithmic Growth

```
int i = 1;
while (i < n) {
    cout << i;
    i = i * 2;
}
```

Problem 4: Mixed Loops

```
for (int i = 0; i < n; i++) {       // Loop A
    cout << i;
}
for (int j = 0; j < n; j++) {       // Loop B
    for (int k = 0; k < n; k++) {
        cout << j << k;
    }
}
```

Problem 5: Trick Question

```
void func(int n) {
    if (n == 1) return;
    for (int i = 0; i < 1000; i++) {
        cout << "Hi";
    }
}
```

Set B: Theoretical Scenarios

1. **Scenario:** You need to look up a student's record by ID. You have 10,000 students.
   - **Case A:** IDs are stored in a sorted array.
   - **Case B:** IDs are stored in a Linked List.
   - *Question:* What is the complexity for both? Which is better?
2. **Scenario:** A recursive function calls itself twice: `T(n) = 2T(n-1) + 1`.
   - *Question:* Does this grow Linearly ($O(n)$) or Exponentially ($O(2^n)$)?
3. **Scenario:** Inserting an element at the **end** of a Singly Linked List.
   - *Question:* What is the complexity if you only have a `Head` pointer? What if you maintain a `Tail` pointer?

Set C: Advanced Challenge Problems

Problem 6: Multiplicative Inner Loop

```
for (int i = 1; i < n; i++) {
    for (int j = 1; j < n; j = j * 2) {
        cout << i << j;
    }
}
```

Problem 7: Recursion + Iteration

```
void recursiveFunc(int n) {
    if (n <= 0) return;
    for (int i = 0; i < n; i++) {
        cout << i;
    }
    recursiveFunc(n - 1);
}
```

Problem 8: The "Square Root" Loop

```
for (int i = 0; i < n; i++) {
    if (i * i > n) break;
```

```
    cout << i;
}
```

Problem 9: Design Challenge
You need a data structure that supports the following operations efficiently:

1. `Add(x)`
2. `Remove(x)`
3. `GetMedian()`
   Constraint: You want `GetMedian()` to be $O(1)$. What combination of data structures would you use? (Hint: Think about sorting/heaps).

Answer Key (Click to Expand)

Set A: Code Snippets

1. $O(n)$: Even though it increments by 2 ($n/2$ iterations), we drop constants.
2. $O(n^2)$: This is the sum of $0 + 1 + 2 \ldots + (n-1)$, which is $n(n-1)/2$.
3. $O(\log n)$: The variable `i` doubles each time ($1, 2, 4, 8 \ldots$), reaching n in $\log_2 n$ steps.
4. $O(n^2)$: Loop A is $O(n)$, Loop B is $O(n^2)$. We take the dominant term ($n^2 > n$).
5. $O(1)$: The loop runs a fixed 1000 times regardless of input n.

Set B: Theory

1. **Sorted Array ($O(\log n)$)** is better than **Linked List ($O(n)$)** because you can use Binary Search on the array.
2. **Exponential ($O(2^n)$)**. This is the classic recurrence for the Tower of Hanoi or naive Fibonacci.
3. **With Head only:** $O(n)$ (must traverse to end). **With Tail:** $O(1)$ (direct access).

Set C: Advanced Challenges
6. $O(n \log n)$: Outer loop runs n times. Inner loop runs $\log n$ times (doubling `j`). Total: $n \times \log n$.
7. $O(n^2)$: The recurrence relation is $T(n) = T(n-1) + n$. This expands to $n + (n-1) + (n-2) \ldots + 1$, which is the arithmetic series sum $\approx n^2/2$.
8. $O(\sqrt{n})$: The condition `i * i > n` is equivalent to `i > sqrt(n)`. The loop breaks when `i` reaches $\sqrt{n}$.
9. **Two Heaps (Max-Heap & Min-Heap)**: Keep the smaller half of numbers in a Max-

Heap and the larger half in a Min-Heap. The median is always at the top of one (or average of both). `Add` is $O(\log n)$, `Median` is $O(1)$.